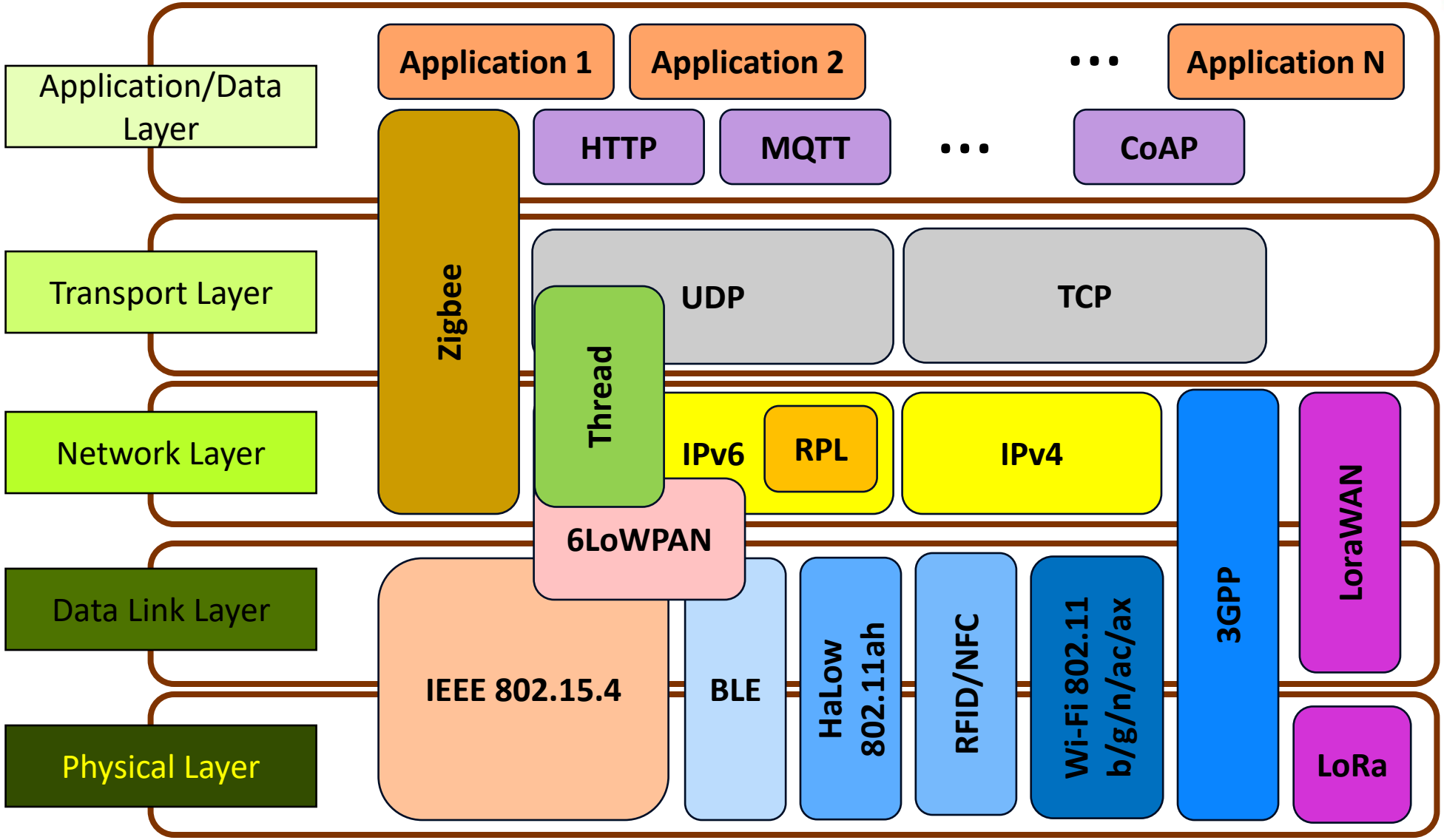
Internet of Things: Protocols and Networks

(CSC2106)

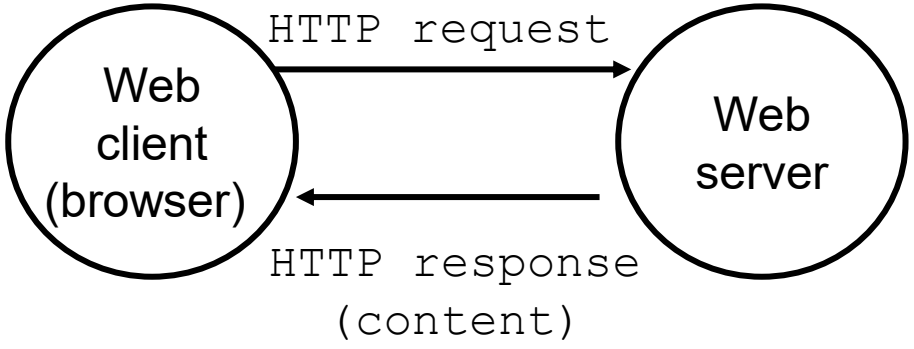
HTTP & REST

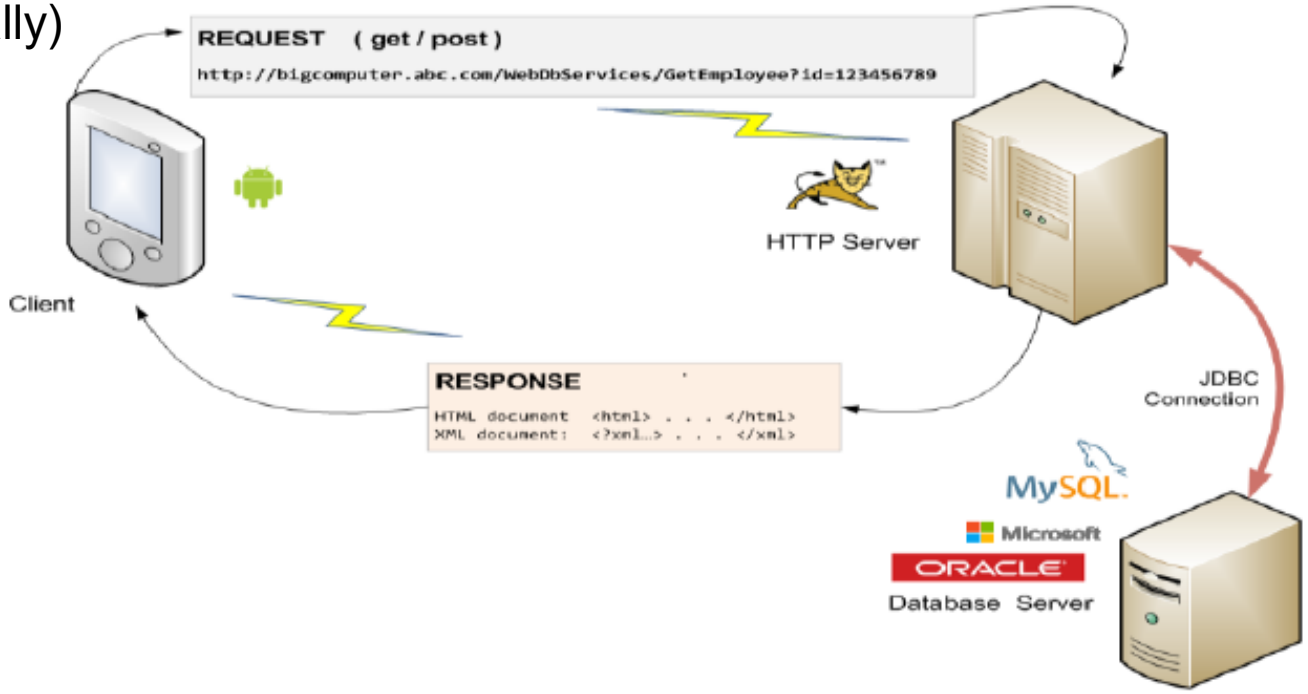
HTTP


Application/Data Layer
Transport Layer
Network Layer
Data Link Layer
Physical Layer
Application 1
Application 2
...
Application N
HTTP
MQTT
...
CoAP
Zigbee
UDP
TCP
Thread
IPv6
RPL
IPv4
6LoWPAN
3GPP
LoraWAN
IEEE 802.15.4
BLE
HaLow 802.11ah
RFID/NFC
Wi-Fi 802.11 b/g/n/ac/ax
LoRa

HyperText Transfer Protocol (HTTP)

- Clients and servers communicate using the HyperText Transfer Protocol (HTTP)
  - Client and server establish TCP connection
  - Client requests content
  - Server responds with requested content
  - Client and server close connection (usually)





Web Content

- Web servers return **content** to clients
  - a sequence of bytes with an associated MIME (Multipurpose Internet Mail Extensions) type
- Example MIME types
  - `text/html` — HTML document
  - `text/plain` — Unformatted text
  - `application/postscript` — Postcript document
  - `image/gif` — Binary image encoded in GIF format
  - `image/jpeg` — Binary image encoded in JPEG format

Static & Dynamic Content

- The content returned in HTTP responses can be either static or dynamic
  - **Static content**: content stored in files and retrieved in response to an HTTP request
    - Examples: HTML files, images, audio clips
  - **Dynamic content**: content produced on-the-fly in response to an HTTP request
    - Example: content produced by a program executed by the server on behalf of the client
- **Bottom line**: all web content is associated with a *file* that is managed by the server

Universal Resource Locator (URL)

- Each *file* managed by a server has a unique name called a *URL*
- URLs for static content:
  - `http://www.singaporetech.edu.sg:80/index.html`
  - `http://www.singaporetech.edu.sg/index.html`
  - `http://www.singaporetech.edu.sg`
    - Identifies a *file* called `index.html`, managed by a web server at `www.singaporetech.edu.sg` that is listening on port 80
- URLs for dynamic content:
  - `http://www.singaporetech.edu.sg:8080/cgi-bin/simi?15432&223`
    - Identifies an executable *file* called `simi`, managed by a web server at `www.singaporetech.edu.sg` that is listening on port 8080, that should be called with two argument strings: `15432` and `223`

HTTP Requests

- HTTP request is a *request line*, followed by zero or more *request headers*
- Request line: `<method> <uri> <version>`
  - `<version>` is HTTP version of request (`HTTP/1.0` or `HTTP/1.1`)
  - `<uri>` is typically URL for proxies and servers
  - `<method>` is either `GET, POST, OPTIONS, HEAD, PUT, DELETE,` or `TRACE`

GET /devices/123/temperature

A **URI** is an **identifier** of a specific resource. Like a page, or book, or a document. A **URL** is **special type of identifier** that also tells you how to access it, such as HTTPs , FTP , etc. .

HTTP Requests

request line

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

- HTTP methods:
  - `GET`: Retrieve static or dynamic content
    - Arguments for dynamic content are in URI
    - Workhorse method (99% of requests)
  - `POST`: Retrieve dynamic content
    - Arguments for dynamic content are in the request body
  - `OPTIONS`: Get server or file attributes
  - `HEAD`: Like `GET` but no data in response body
  - `PUT`: Write a file to the server
  - `DELETE`: Delete a file on the server
  - `TRACE`: Echo request in response body
    - Useful for debugging

HTTP Responses

- HTTP response is a *response line* followed by zero or more *response headers*
- Response line:
- `<version> <status code> <status msg>`
  - <version> is HTTP version of the response
  - <status code> is numeric status
  - <status msg> is corresponding English text
    - 200 OK - Request was handled without error
    - 403 ForbiddenServer lacks permission to access file
    - 404 Not found Server couldn't find the file
- Response headers: `<header name>: <header data>`
  - Provide additional information about response
  - `Content-Type:` MIME type of content in response body
  - `Content-Length:` Length of content in response body

Example:

```
{
 "temperature": 26.5,
 "unit": "Celsius"
}
```

HTTP Responses (example)

version

status code & status msg

header name

header data

response line

response headers

response body

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Connection: Closed

<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

Example of an HTTP Transaction from an IoT Device via Telnet

Client (IoT Device) Action

```
iot-device> telnet api.iotserver.com 80 # IoT opens connection to server

Trying 192.168.1.100...                 # Server IP resolution

Connected to api.iotserver.com.

Escape character is '^]'.


POST /devices/123/temperature HTTP/1.1  # IoT device sends HTTP
                                        # POST request

Host: api.iotserver.com                 # Required HTTP/1.1
                                        # HOST header

Content-Type: application/json          # Specifies JSON data format

Content-Length: 45                      # Size of payload

                                        # Empty line terminates headers

{ "device_id": "123", "temperature": "26.5" }  # JSON payload with sensor data
```

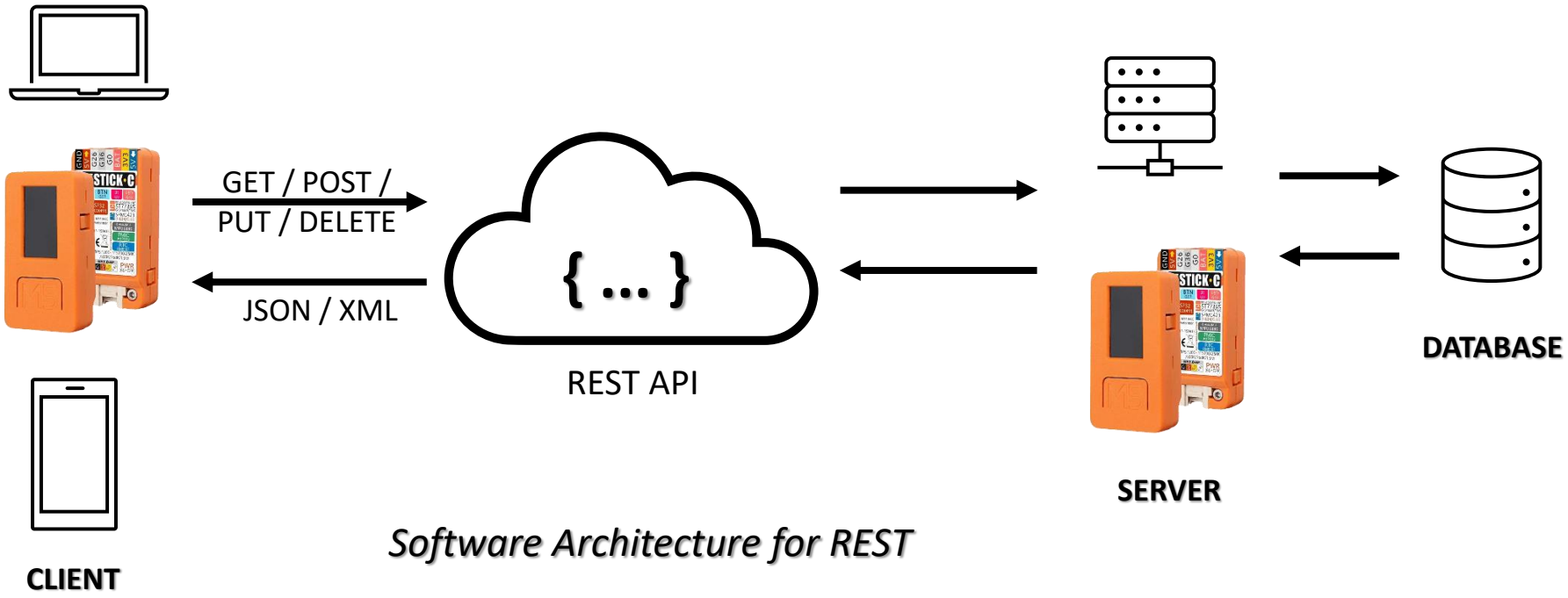
Server (Cloud API) Response

```
HTTP/1.1 201 Created                    # Server response line
                                        # Success)

Date: Tue, 12 Mar 2024 10:15:30 GMT     # Timestamp of response

Server: IoT-Cloud/1.0                   # Server software

Content-Type: application/json          # Response format

Content-Length: 35                      # Response payload size

Connection: close                       # Server closes connection


{ "status": "success", "id": "98765" }  # Response body confirming
                                        # data saved
```

{REST:API}

REpresentational State Transfer (REST)

- First introduced by Roy Fielding in 2000.
- Web standards-based architecture and uses HTTP Protocol
- REST is not a protocol but an architectural style
- Every component is a resource that is accessed via HTTP protocols.
- REST can adopt several standards, e.g. HTTP, URL, XML, HTML, GIF, JPEG, etc



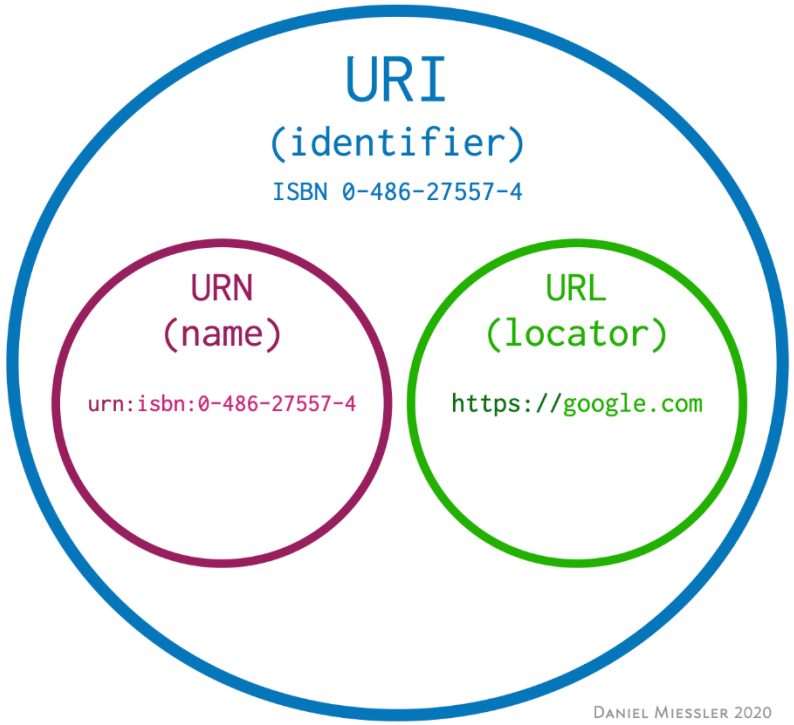
Software Architecture for REST

https://blog.ndepend.com/rest-vs-restful/

REST & HTTP

- The motivation for REST was to take advantage of the success of the Web
  - URI Addressable resources → URL
  - HTTP Protocol
  - Request → Response → Display Response

- Make use of the HTTP protocol beyond POST & GET
  - HTTP PUT, HTTP DELETE

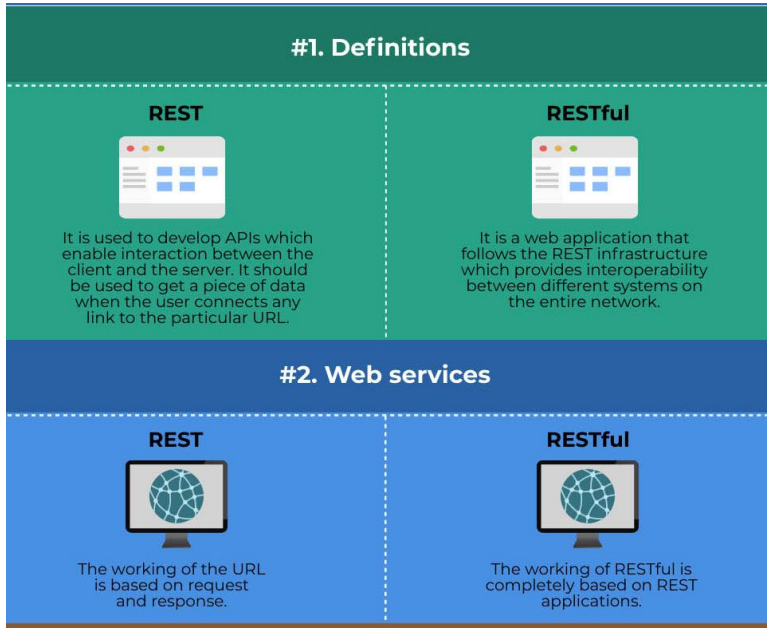


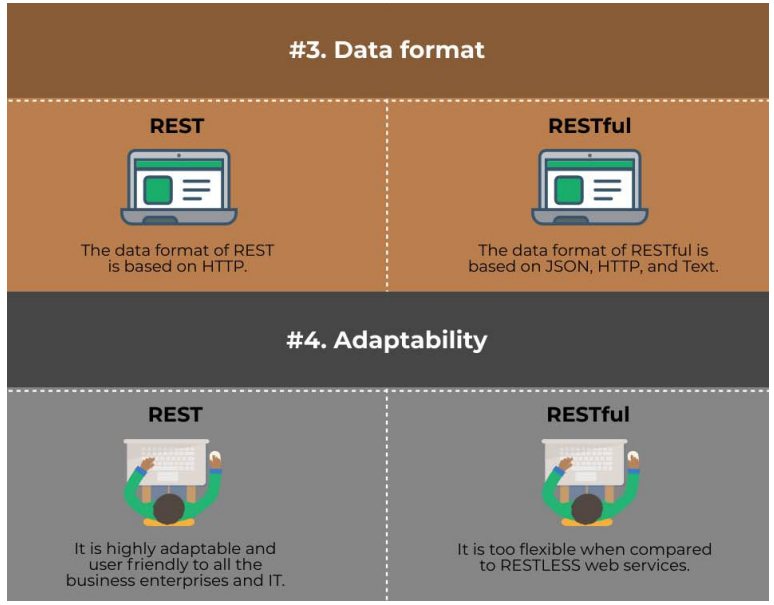
A URI is an identifier.
A URL is an identifier that tells you how to get to it.

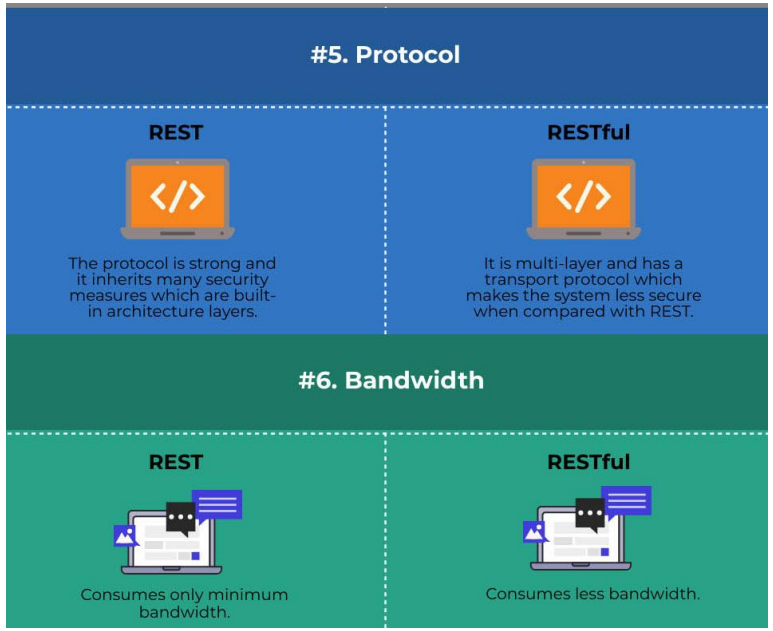
Six Architectural Constraints of REST

1. **Client-Server Architecture** - The client and server act independently.
2. **Statelessness** - The server does not record the state of the client.
3. **Cacheable** - The server marks whether data is cacheable.
4. **Layered System** - The application behaves the same regardless of any intermediaries between the client and server.
5. **Uniform Interface** - The client and server interact in a uniform and predictable way. An important aspect of this is that the server exposes resources.
6. **Code on Demand (optional)** – It allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented. Allowing features to be downloaded after deployment improves system extensibility.

REST vs RESTful







https://www.educba.com/rest-vs-restful/

Resources

- The key abstraction of information in REST is a resource
- A resource is a conceptual mapping to a set of entities
  - Any information that can be named can be a resource: a document or image, a temporal service (e.g., “today's weather in Berlin”), a collection of other resources, a non-virtual object (e.g., a person), etc.
- Represented with a global identifier (URI in HTTP)
  - http://www.boeing.com/aircraft/747

Naming Resources

- REST uses URI to identify resources

  - http://localhost/books/ → **Retrieves all books**
  - http://localhost/books/ISBN-0011 → **Retrieves a specific book**
  - http://localhost/books/ISBN-0011/authors → **Retrieves the authors of that specific book**

  - http://localhost/classes
  - http://localhost/classes/cs2650
  - http://localhost/classes/cs2650/students

- As you traverse the path from more generic to more specific, you are navigating the data

REST Verbs

Create
Read
Update
Delete

Verb	Objective	Usage	Multiple Request	Cache
GET	Retrieve items from resource	Links	Yes	Yes
POST	Create new item in resource	Forms	No	No
PUT	Replace existing item in resource	Forms	Yes	No
PATCH	Update existing item in resource	Forms	No/Yes	No
DELETE	Delete existing item in resource	Forms/Links	Yes	No

GET

- How clients ask for the information they seek
- Issuing a GET request transfers the data from the server to the client in some representation
- GET http://localhost/books
  - Retrieve all books

Response: Returns book data.

- GET http://localhost/books/ISBN-0011021
  - Retrieve book identified with ISBN-0011021
- GET http://localhost/books/ISBN-0011021/authors
  - Retrieve authors for book identified with ISBN-0011021

Safe and idempotent (doesn't change the resource).

HTTP Request: Client → Server

```
GET /books/ISBN-0011021 HTTP/1.1
Host: localhost
```

HTTP Response: Server → Client

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "isbn": "0011021",
  "title": "Sample Book",
  "authors": ["Author A", "Author B"]
}
```

POST, PUT & PATCH

- HTTP POST creates a resource ***Not idempotent*** *(multiple requests create multiple entries).*
- HTTP PUT updates the (entire) resource ***Idempotent*** *(multiple requests produce the same result).*

- POST http://localhost/books/ **Response: 201 Created with the new book details.**
  - Content: {“isbn”: “111-456-789”, “title”: “The Boring Two”, “authors”: …}
  - Creates a new book with given properties

- PUT http://localhost/books/isbn-111 **Response: 200 OK with updated book details.**
  - Content: {“isbn”: “123-456-789”, “title”: “The Boring One”, “authors”: …}
  - Update book identified by isbn-111 with submitted properties

- PATCH http://localhost/books/isbn-111 **Response: 200 OK with updated book title.**
  - Content: {“title”: “The Boring One”}
  - Update book identified by isbn-111 with submitted properties

POST Example:
Create a new resource

Request:

```
POST /books HTTP/1.1
Host: localhost
Content-Type: application/json

{
  "isbn": "111-456-789",
  "title": "The Boring Two",
  "authors": ["Author A", "Author B"]
}
```

Response:

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "isbn": "111-456-789",
  "title": "The Boring Two",
  "authors": ["Author A", "Author B"]
}
```

PUT Example:
Update (replace) an entire resource

Request:

```
PUT /books/isbn-111 HTTP/1.1
Host: localhost
Content-Type: application/json

{
  "isbn": "123-456-789",
  "title": "The Boring One",
  "authors": ["Author C"]
}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "isbn": "123-456-789",
  "title": "The Boring One",
  "authors": ["Author C"]
}
```

PATCH Example:
Update part of a resource

Request:

```
PATCH /books/isbn-111 HTTP/1.1
Host: localhost
Content-Type: application/json

{
  "title": "The Boring One"
}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "isbn": "123-456-789",
  "title": "The Boring One",
  "authors": ["Author C"]
}
```

DELETE

HTTP Request: Client → Server

```
DELETE /books/isbn-111 HTTP/1.1
Host: localhost
```

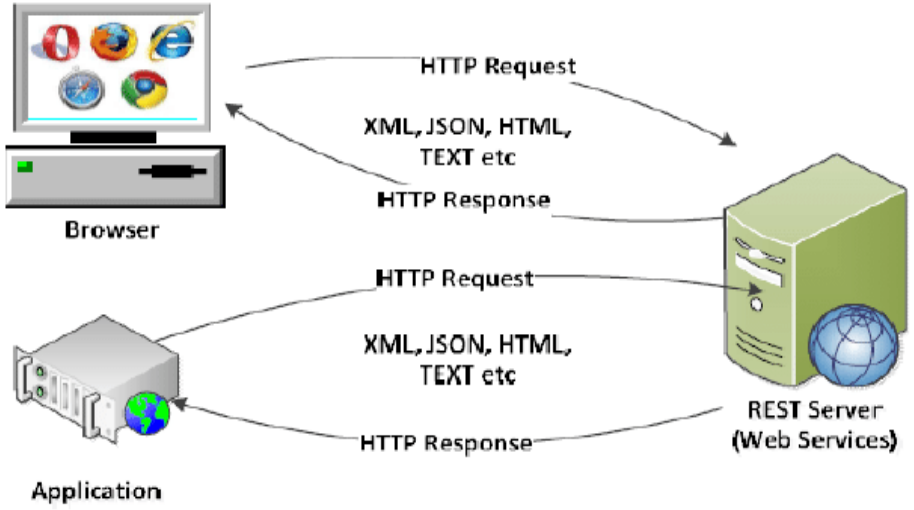
HTTP Response: Server → Client

```
HTTP/1.1 204 No Content
```

Idempotent → *deleting twice has the same result*

Representations

- How data is represented or returned to the client for presentation.
- Typical formats:
  - JavaScript Object Notation (**JSON**)
  - Extensible Markup Language (**XML**)




```
{ "id": 1, "name": "John Doe", "email": "john@example.com" }
```

```
<user>
  <id>1</id>
  <name>John Doe</name>
  <email>john@example.com</email>
</user>
```

- The above shown are <u>not</u> the <u>resource</u> itself.
- They are just a way to <u>represent</u> the <u>resource</u> which is stored somehow in your server.

RESTful for IoT

Best Practices

- Use clear, hierarchical resource names (e.g., /devices/{id}/status)
- Keep APIs stateless
- Prefer lightweight data formats (JSON)
- Secure endpoints (authentication, HTTPS)

Security Note

- Exposing device controls via HTTP can be risky
- Always use authentication and HTTPS
- Limit access and validate all inputs

Limitations of HTTP for IoT

- **High Overhead**
  - Large headers and text-based format increase message size and complexity, making HTTP inefficient for small IoT payloads..
- **Request-Response Model**
  - Synchronous communication is not ideal for real-time or event-driven IoT.
  - Devices must poll for updates, raising traffic and energy use.
- **Latency and Connection Management**
  - Each request needs a new TCP connection, causing extra latency and power drain.
  - HTTP/2 helps, but is still less efficient than IoT-specific protocols.
- **Not Optimized for Constrained Networks**
  - Protocols like LoRa and Zigbee don’t support HTTP due to its overhead.
  - HTTP suits high-bandwidth, always-on networks, not low-power or lossy IoT environments..
- **Security Concerns**
  - Data is sent in plaintext unless using HTTPS, which adds computational and bandwidth overhead, challenging for resource-limited devices.

Comparison Table: HTTP, MQTT, and CoAP

Feature	HTTP	MQTT	CoAP
Model	Request-Response	Publish-Subscribe	Request-Response
Overhead	High (text headers)	Low (2-byte header)	Very Low (binary, compact)
Bandwidth	High	Low	Very Low
Suitability	Web, dashboards	Sensors, telemetry, alerts	Constrained devices, M2M
Power Usage	High	Low	Very Low
Security	HTTPS (TLS)	TLS, username/password	DTLS
Reliability	Reliable (TCP)	QoS Levels 0–2	Confirmable messages
Scalability	Moderate	High	High

https://fordewind.io/choosing-the-right-iot-protocol-mqtt-coap-or-http/
https://www.javacodegeeks.com/2025/09/lightweight-iot-messaging-comparing-mqtt-vs-http-for-constrained-devices.html

Summary – HTTP & REST in IoT

- Definitions
  - HTTP is the foundation of web communication, using **request-response** interactions.
  - REST is an **architectural style** that leverages HTTP for scalable and structured APIs.
- IoT relevance
  - IoT devices use **RESTful APIs** to allow remote monitoring, device control, and data retrieval.
  - Well-suited for **high-bandwidth**, internet-connected IoT applications (e.g., smart homes, industrial dashboards).
- Challenges
  - **High overhead:** (Relatively large) HTTP headers make it inefficient for low-power devices.
  - **Request-response model:** Not ideal for real-time or event-driven IoT applications.
  - **Latency:** REST APIs require a new connection for each request, increasing delays.
  - **Not optimised for constrained networks**: LoRa and Zigbee don't directly support HTTP.
- Overcoming These Challenges
  - **MQTT** (Message Queuing Telemetry Transport)
    - A lightweight protocol designed for low-bandwidth, real-time communication via publish-subscribe messaging.
  - **CoAP** (Constrained Application Protocol)
    - A REST-based protocol optimised for low-power, constrained devices with a smaller message format.